



JESSICA • CARMEN • YANTING • EMMA

“Just Connect Your Experiences,
Analysis, and Insights”

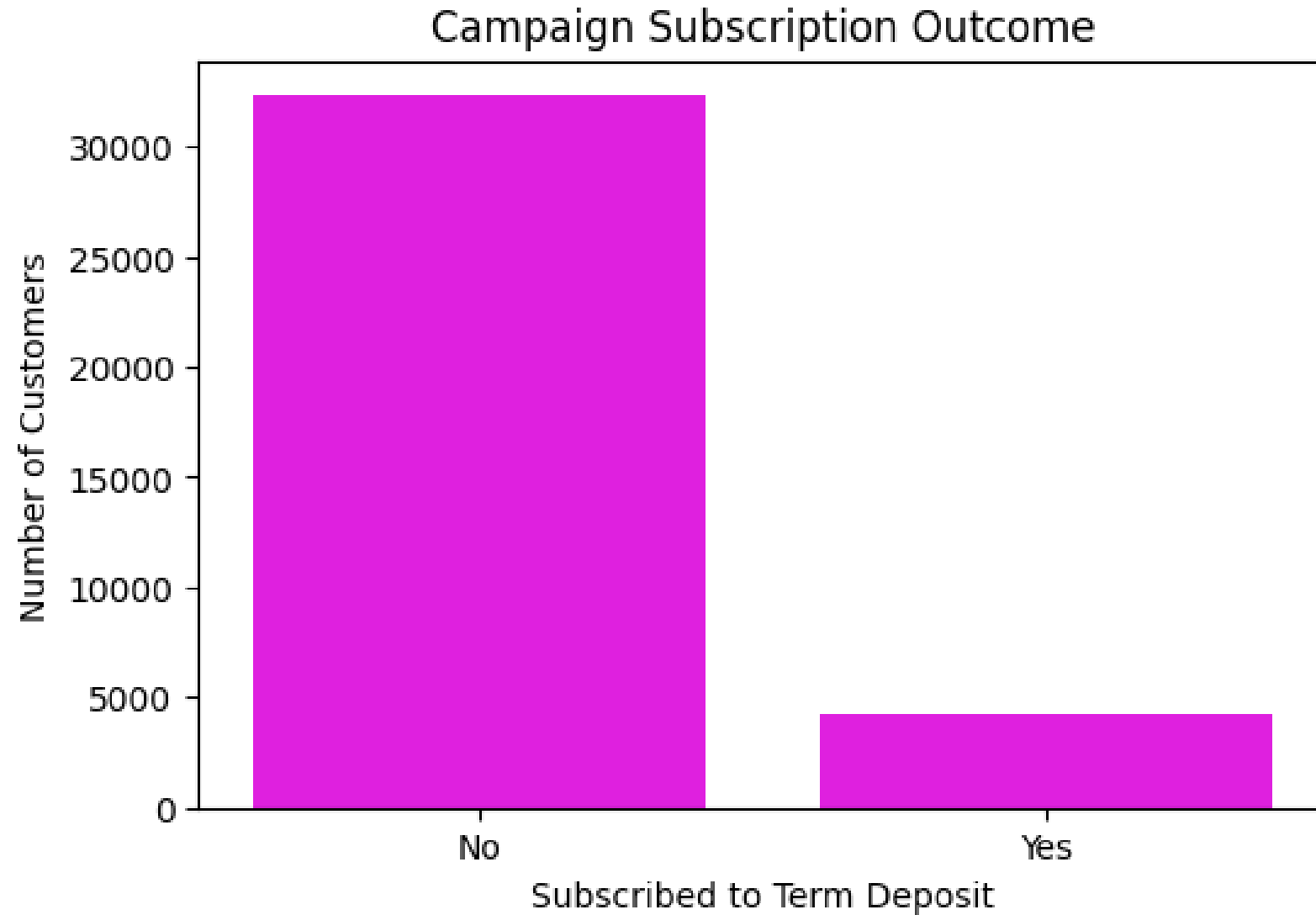
Bank Marketing Dataset

By: Jessica Valenzuela, Carmen Bahr,
Yanting Wang, & Emma Schupbach

Business Context

- ▶ Our dataset is a bank marketing dataset collected from a direct marketing campaign conducted by a Portuguese bank. The dataset includes client information, contact type, previous campaign outcomes, and social and economic factors.

Campaign Analysis



Business Problem & Objectives

- ▶ The bank spends significant resources contacting customers through telemarketing campaigns. However, many customers decline the offer.
- ▶ The business objective is to identify which customer characteristics are associated with a higher likelihood of subscribing to a term deposit so that marketing efforts can be focused on the most promising prospects.

Dataset Overview

- ▶ The dataset includes:
 - Demographic information
 - Financial information
 - Campaign details
 - Economic indicators
- ▶ The target variable is **y**, which indicates whether the customer subscribed to a term deposit:
 - **yes** → subscribed
 - **no** → not subscribed
- ▶ This is a binary classification dataset containing both numerical and categorical variables.

age
job
marital
education
default
housing
loan
contact
month
day_of_week
campaign
pdays
previous
poutcome
emp.var.rate
cons.price.idx
cons.conf.idx
euribor3m
nr.employed
y

Describe data set

- These are all of the features that were numerical prior to changing them to binary or scaling them. By describing the data it gave us a better understanding of how it lies before we get into cleaning it.

	count	mean	std	min	25%	50%	75%	max
age	41188.0	40.024060	10.421250	17.000	32.000	38.000	47.000	98.000
campaign	41188.0	2.567593	2.770014	1.000	1.000	2.000	3.000	56.000
pdays	41188.0	962.475454	186.910907	0.000	999.000	999.000	999.000	999.000
previous	41188.0	0.172963	0.494901	0.000	0.000	0.000	0.000	7.000
emp.var.rate	41188.0	0.081886	1.570960	-3.400	-1.800	1.100	1.400	1.400
cons.price.idx	41188.0	93.575664	0.578840	92.201	93.075	93.749	93.994	94.767
cons.conf.idx	41188.0	-40.502600	4.628198	-50.800	-42.700	-41.800	-36.400	-26.900
euribor3m	41188.0	3.621291	1.734447	0.634	1.344	4.857	4.961	5.045
nr.employed	41188.0	5167.035911	72.251528	4963.600	5099.100	5191.000	5228.100	5228.100

Remove Duplicate Values

- We checked for duplicate rows and found that there were 1784 of them. So, we decided to drop them.

```
# Check for duplicate rows  
print("Number of duplicated rows:", df.duplicated().sum())
```

```
Number of duplicated rows: 1784
```

```
# Delete duplicate rows  
df = df.drop_duplicates()  
print("New dataset shape:")  
print(df.shape)
```

```
New dataset shape:  
(39404, 20)
```

Dropping Missing Values

- ▶ We checked for missing values and found that there were some missing from our y column.

```
# Drop rows where y is missing  
df = df.dropna(subset=['y'])
```

Outliers

- ▶ We checked for outliers in our database by calculating the interquartile range and found that we had some in the age and campaign columns
- ▶ We then decided to drop these outliers so that they didn't skew our data.

```
#Checking
cols_to_check = ['age', 'campaign']

# Keep a record of your original row count
original_rows = len(df)

for col in cols_to_check:
    # Calculate Q1 (25th percentile) and Q3 (75th percentile)
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    # Calculate the IQR range
    IQR = Q3 - Q1

    # Define your upper and lower boundaries
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Filter the dataframe to only keep rows within the boundaries
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

print(f"Dropped {original_rows - len(df)} statistical outlier rows.")
```

Dropped 2384 statistical outlier rows.

Dealing with Categorical Data

- We then changed all the categorical values into dummy variables so that our model could more easily read the data.

```
# Change categorical to binary
df = pd.get_dummies(
    data=df,
    columns=['job', 'education', 'marital', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'y', 'outcome'],
    drop_first=True
)
bool_cols = df.select_dtypes(include='bool').columns

df[bool_cols] = df[bool_cols].astype(int)
```

	age	campaign	pdays	previous	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed	job_blue-collar	...
0	56	1	999	0	1.1	93.994	-36.4	4.857	5191.0	0	...
1	57	1	999	0	1.1	93.994	-36.4	4.857	5191.0	0	...
2	37	1	999	0	1.1	93.994	-36.4	4.857	5191.0	0	...
3	40	1	999	0	1.1	93.994	-36.4	4.857	5191.0	0	...
4	56	1	999	0	1.1	93.994	-36.4	4.857	5191.0	0	...

Scaling the Data

- We then scaled the numerical variables using MinMax Scaling so they were on a similar range and easier to compare.

```
# Scale appropriate columns
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(-1, 1))

cols_to_scale = [
    'campaign', 'previous', 'emp.var.rate', 'cons.price.idx', 'nr.employed', 'pdays', 'cons.conf.idx', 'euribor3m', 'age']

existing_cols = [c for c in cols_to_scale if c in df.columns]

df[existing_cols] = scaler.fit_transform(df[existing_cols])

df.head()
```

	age	campaign	pdays	previous	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed	job_blue-collar
0	0.500000	-1.0	1.0	-1.0	0.875	0.397506	0.205021	0.914759	0.719471	0
1	0.538462	-1.0	1.0	-1.0	0.875	0.397506	0.205021	0.914759	0.719471	0
2	-0.230769	-1.0	1.0	-1.0	0.875	0.397506	0.205021	0.914759	0.719471	0
3	-0.115385	-1.0	1.0	-1.0	0.875	0.397506	0.205021	0.914759	0.719471	0
4	0.500000	-1.0	1.0	-1.0	0.875	0.397506	0.205021	0.914759	0.719471	0

Hypothesis Testing

- ▶ Test 1: Housing vs Subscription
- ▶ Test 2: Previous Campaign Outcome vs Subscription
- ▶ The results suggest that customer housing loan status and previous campaign outcomes are related to subscription behavior.

Hypothesis Testing

- We chose the Chi-Square hypothesis test because it tests the relationships between categorical variables. Both tests showed a p-value that is less than 0.05, meaning that they show a statistically significant relationship to subscription success.

```
from scipy.stats import chi2_contingency

table = pd.crosstab(df['housing_yes'], df['y_yes'])

chi2, p, dof, expected = chi2_contingency(table)

print("Chi-Square:", chi2)
print("p-value:", p)
```

```
Chi-Square: 4.832797582735421
p-value: 0.027923285383957108
```

```
table = pd.crosstab(df['outcome_success'], df['y_yes'])

chi2, p, dof, expected = chi2_contingency(table)

print("Chi-Square:", chi2)
print("p-value:", p)
```

```
Chi-Square: 3583.4148147292713
p-value: 0.0
```

Model Analysis

Model Type	Accuracy	Precision	Recall	F1
Random Forest	0.8866	0.5593	0.2784	0.3717
Logistical Regression	0.8953	0.7035	0.2262	0.3423
Decision Tree	0.8336	0.3255	0.3548	0.3395

Confusion Matrix Comparison

0 = No subscription

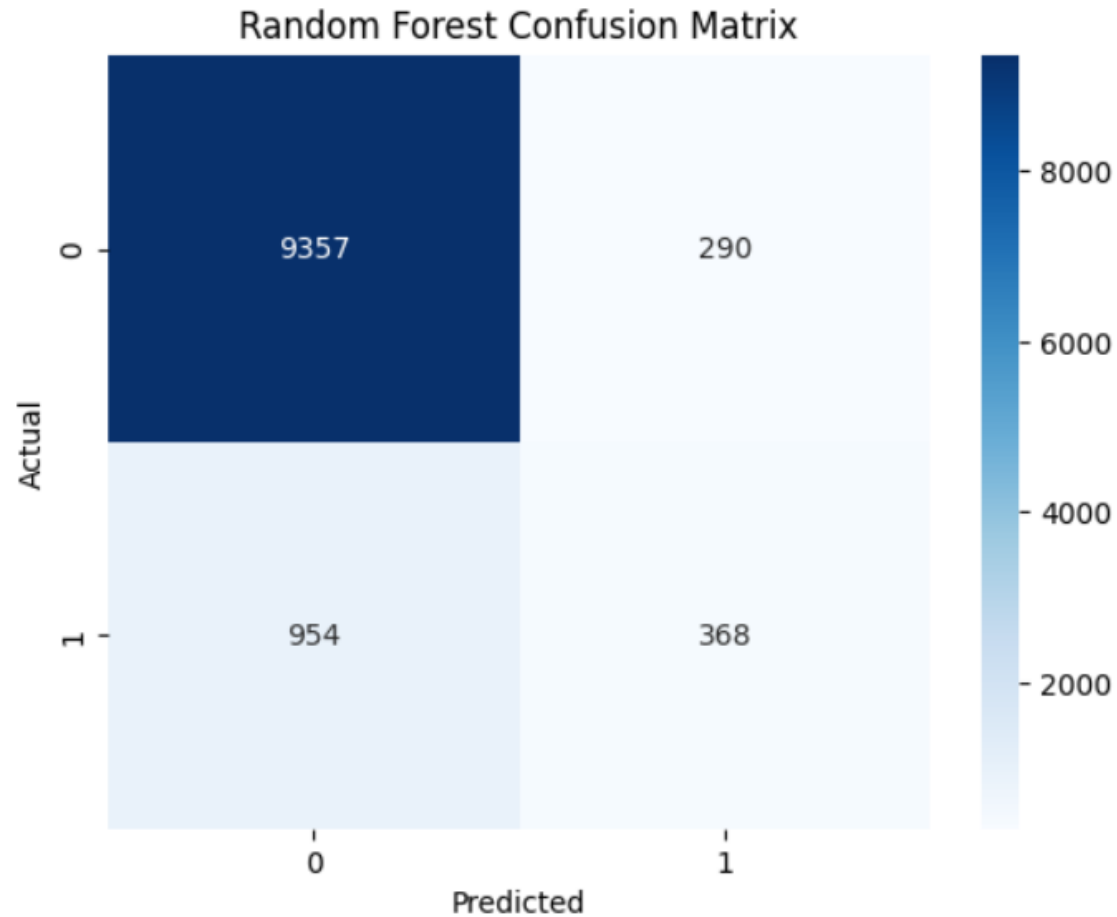
1 = Subscription

The model accurately predicted no subscription 9357 times.

It inaccurately predicted people would subscribe when they actually didn't 290 times.

It accurately predicted subscriptions 368 times.

It inaccurately predicted people wouldn't subscribe when they actually did 954 times.

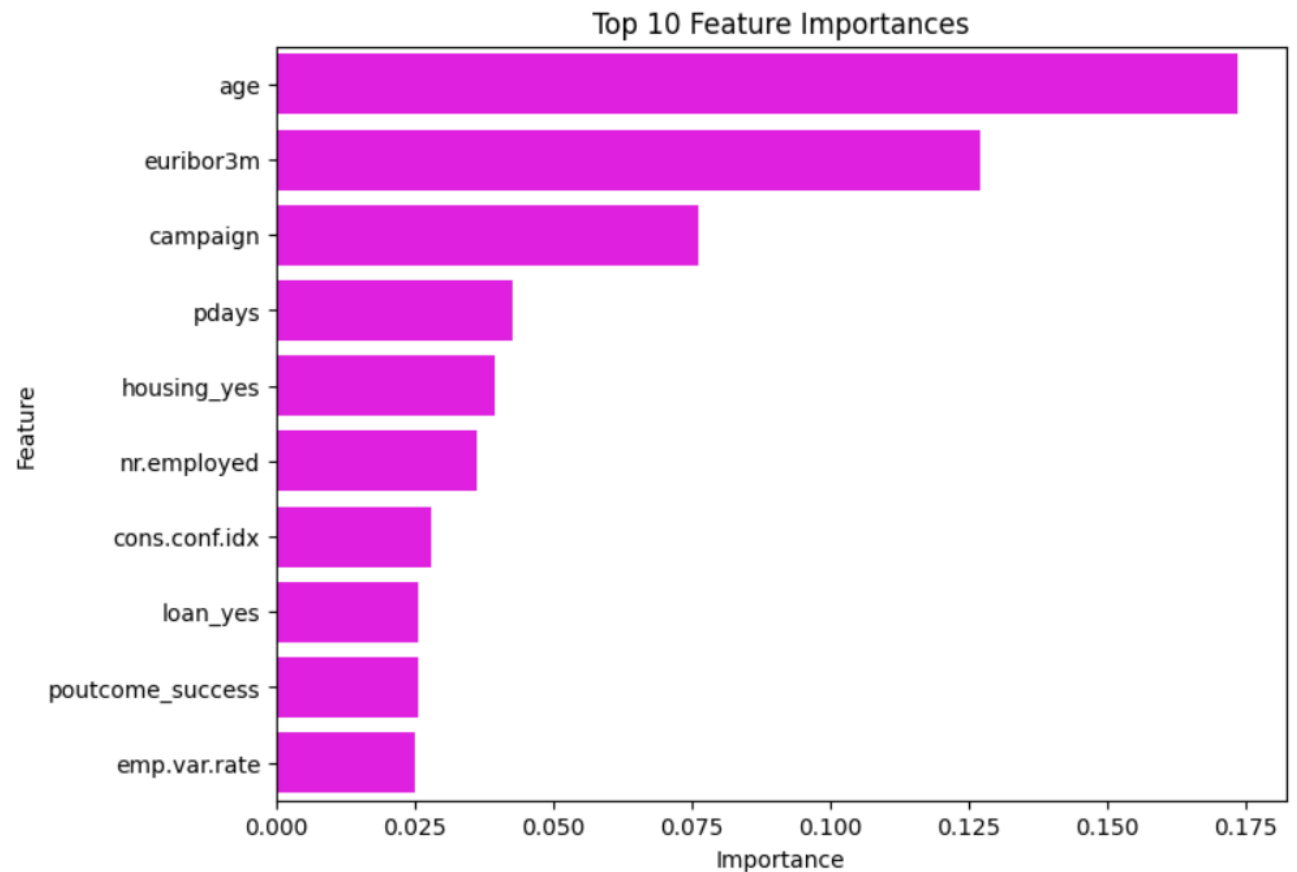


Feature Importance

► Top 10 features

```
importance = pd.DataFrame({  
    'Feature': X.columns,  
    'Importance': rf_model.feature_importances_  
})  
  
importance = importance.sort_values(  
    by='Importance',  
    ascending=False  
)  
  
importance.head(10)
```

	Feature	Importance
0	age	0.173628
7	euribor3m	0.127113
1	campaign	0.076321
2	pdays	0.042545
33	housing_yes	0.039363
8	nr.employed	0.036071
6	cons.conf.idx	0.027963
35	loan_yes	0.025543
51	poutcome_success	0.025521
4	emp.var.rate	0.025072



Model Feature Inclusion Comparison

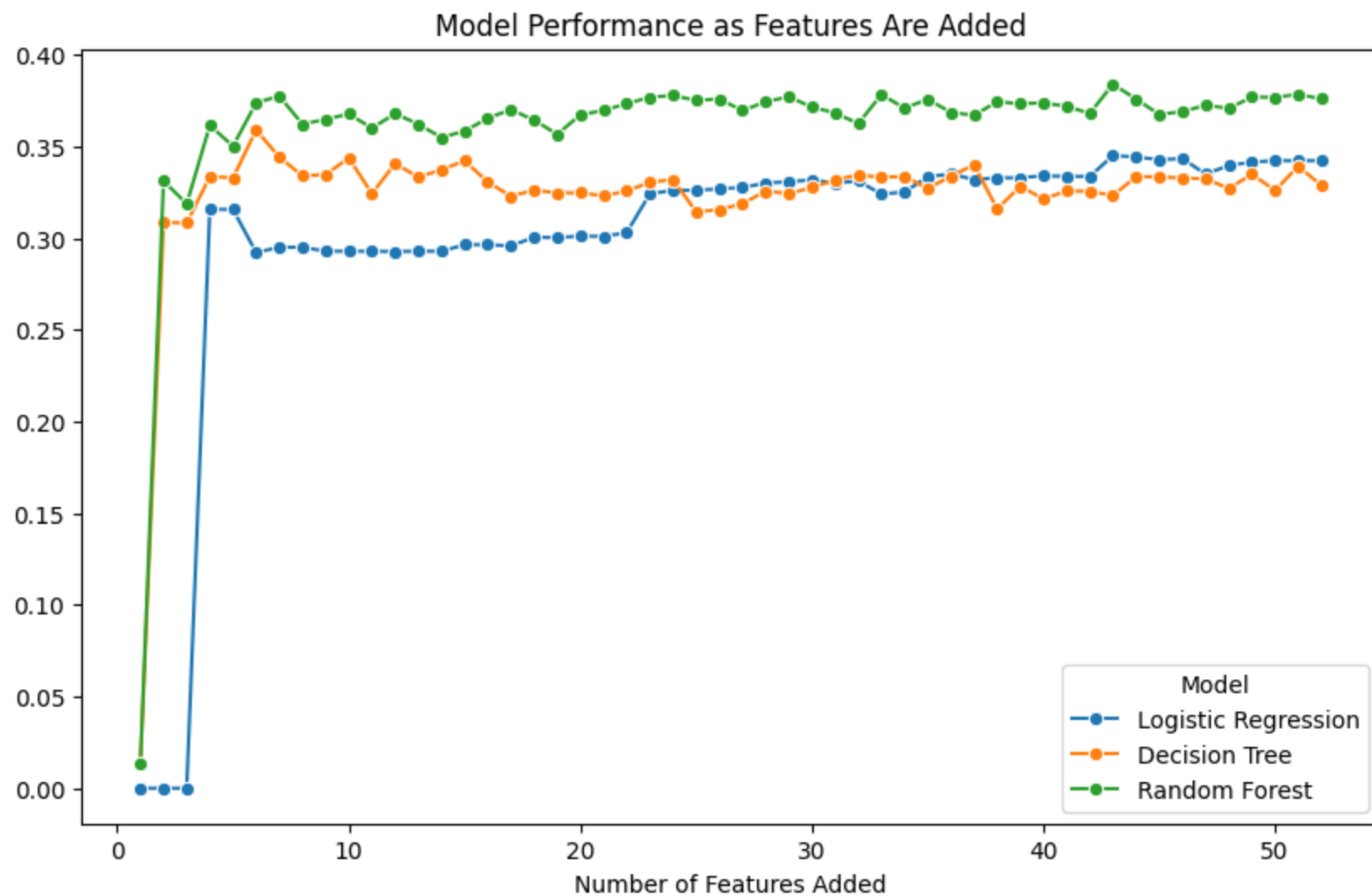
- ▶ Tested model performance by adding features one at a time based on feature importance
- ▶ Compared Logistic regression, Decision Tree, and Random Forest using F1 score
- ▶ Random Forest achieved the best performance with **43 of 52 features (F1= 0.384)**

	Number of Features	Model	Accuracy	Precision	Recall	F1 Score	Features Used
0	1	Logistic Regression	0.879479	0.000000	0.000000	0.000000	[age]
1	1	Decision Tree	0.878840	0.360000	0.006808	0.013363	[age]
2	1	Random Forest	0.878840	0.360000	0.006808	0.013363	[age]
3	2	Logistic Regression	0.879479	0.000000	0.000000	0.000000	[age, euribor3m]
4	2	Decision Tree	0.878658	0.492537	0.224660	0.308571	[age, euribor3m]
...	...	...	...	...	...	...	...
151	51	Decision Tree	0.834169	0.326345	0.353253	0.339266	[age, euribor3m, campaign, pdays, housing_yes,...]
152	51	Random Forest	0.888322	0.574730	0.282148	0.378488	[age, euribor3m, campaign, pdays, housing_yes,...]
153	52	Logistic Regression	0.895250	0.703529	0.226172	0.342301	[age, euribor3m, campaign, pdays, housing_yes,...]
154	52	Decision Tree	0.832163	0.317382	0.341150	0.328837	[age, euribor3m, campaign, pdays, housing_yes,...]
155	52	Random Forest	0.887136	0.563444	0.282148	0.376008	[age, euribor3m, campaign, pdays, housing_yes,...]

156 rows x 7 columns

	Number of Features	Model	Accuracy	Precision	Recall	F1 Score	Features Used
128	43	Random Forest	0.888869	0.578387	0.287443	0.384032	[age, euribor3m, campaign, pdays, housing_yes,...]
152	51	Random Forest	0.888322	0.574730	0.282148	0.378488	[age, euribor3m, campaign, pdays, housing_yes,...]
98	33	Random Forest	0.887957	0.570992	0.282905	0.378351	[age, euribor3m, campaign, pdays, housing_yes,...]
71	24	Random Forest	0.885040	0.543262	0.289713	0.377898	[age, euribor3m, campaign, pdays, housing_yes,...]
20	7	Random Forest	0.876744	0.482353	0.310136	0.377532	[age, euribor3m, campaign, pdays, housing_yes,...]
86	29	Random Forest	0.887410	0.565809	0.282905	0.377206	[age, euribor3m, campaign, pdays, housing_yes,...]
146	49	Random Forest	0.887319	0.564955	0.282905	0.377016	[age, euribor3m, campaign, pdays, housing_yes,...]
68	23	Random Forest	0.884857	0.541844	0.288956	0.376912	[age, euribor3m, campaign, pdays, housing_yes,...]
149	50	Random Forest	0.886954	0.561377	0.283661	0.376884	[age, euribor3m, campaign, pdays, housing_yes,...]
155	52	Random Forest	0.887136	0.563444	0.282148	0.376008	[age, euribor3m, campaign, pdays, housing_yes,...]

Feature Comparison Visual



Recommendation 1

- ▶ Prioritize customers with previous campaigns success
 - Previous campaign success was one of the strongest predictors of subscription
 - Create priority call lists using customers who responded positively to prior campaigns
 - Allocate more follow-up resources to these customers first

Recommendation 2

- ▶ Limit excessive contact attempts
 - The number of campaign contacts was one of the most important predictors in the model.
 - A statistical test showed a significant difference in campaign contacts between subscribers and non-subscribers.
 - Focus outreach on high-potential customers rather than repeatedly contacting unresponsive prospects.

```
from scipy.stats import ttest_ind

no_group = df[df['y_yes'] == 0]['campaign']
yes_group = df[df['y_yes'] == 1]['campaign']

t_stat, p_value = ttest_ind(no_group, yes_group, equal_var=False)

print("Mean campaign contacts - No:", no_group.mean())
print("Mean campaign contacts - Yes:", yes_group.mean())
print("t-statistic:", t_stat)
print("p-value:", p_value)
```

```
... Mean campaign contacts - No: -0.558486124876115
Mean campaign contacts - Yes: -0.6441740758072064
t-statistic: 11.117496199625412
p-value: 1.9968039042068163e-28
```

Recommendation 3

- ▶ Align campaign timing with favorable economic conditions
 - Economic indicators such as euribor3m, nr.employed, emp.var.rate, and cons.conf.idx were among the most influential predictors
 - Increase marketing activity during periods when economic conditions historically align with higher subscription rates
 - Use economic trends to help determine campaign timing and resource allocation

Conclusion

- ▶ Random Forest was the strongest-performing model, achieving the highest F1 score and best overall balance between precision and recall.
- ▶ Previous campaign success, campaign activity, and economic indicators were among the most important factors influencing customer subscription decisions.
- ▶ The model can help the bank improve marketing efficiency by focusing outreach on customers most likely to subscribe.

Thank You!

